

Data Governance Copilot

Interview Question Bank

Topic 11 — MCP Protocol & Server

12 questions · 4 sections · stdio vs SSE · 10 Tools · approve_action HITL · MCP Client · Auth · Protocol Risk

Foundational Core concepts **Intermediate** Design decisions **Advanced** Deep internals **Gotcha** Real bugs / traps

MCP Protocol Fundamentals

Q01**Foundational**

What is the Model Context Protocol (MCP) and why did you implement a standalone MCP server on top of your existing FastAPI API?

Hint: Think about what MCP enables that a REST API alone doesn't.

Key points to cover:

- MCP is an open protocol (Anthropic-initiated) that standardises how AI assistants discover and call tools exposed by external servers
- It defines a client-server model: MCP client (Claude Desktop, Claude.ai, any LLM host) discovers tools from an MCP server and can invoke them during inference
- Without MCP: Claude Desktop has no native way to query your governance system — you'd need a custom plugin or extension per client
- With MCP: Claude Desktop reads your `claude_desktop_config_example.json`, connects to your server, and your 10 tools appear in Claude's tool palette automatically
- FastAPI serves HTTP clients (browsers, curl, code) — MCP serves AI assistant clients (Claude Desktop, future MCP-compatible agents)
- They share the same underlying agents and graph code via `src/` imports — MCP server is a new interface, not a new system
- Standalone server (`src/mcp_server/server.py`): runs as a separate process — can be used without the FastAPI server running; independently deployable
- Portfolio value: MCP server demonstrates cutting-edge AI integration capability — very few candidates have built a production MCP server

Q02**Foundational**

Describe the two MCP transport modes — stdio and SSE. When is each used and how does your project configure the switch?

Hint: These are fundamentally different communication channels — explain the mechanics of each.

Key points to cover:

- stdio transport: MCP server communicates via `stdin/stdout` — Claude Desktop launches your server as a subprocess and exchanges JSON-RPC messages over pipes
- stdio is the default (`TRANSPORT=stdio`) — used for local Claude Desktop integration where server runs on the same machine
- SSE transport: server listens on HTTP (`MCP_PORT=8002`) and pushes events via Server-Sent Events — client connects over network
- SSE is used for remote/web clients — `compose.yml` `mcp_server` service uses `TRANSPORT=sse`, port 8002
- Config switch: `server.py` reads `TRANSPORT` env var at startup; if 'sse' → start HTTP server with SSE handler; if 'stdio' (default) → use stdio transport
- Claude Desktop config (`claude_desktop_config_example.json`): specifies command to launch the server process — Claude Desktop manages the subprocess lifecycle
- SSE in production: remote MCP clients connect to the SSE endpoint; authentication (API key or VPC restriction) must be added
- stdio advantage: zero network overhead, no port to expose, process isolation — ideal for local dev; SSE advantage: multiple remote clients, no subprocess management by client

```
# claude_desktop_config_example.json { "mcpServers": { "data-governance-copilot": { "command":  
  "python", "args": ["-m", "src.mcp_server.server"], "env": { "ENABLE MOCK": "true", "TRANSPORT":  
    "stdio" } } } } # SSE (compose.yml) mcp_server: command: python -m src.mcp_server.server  
environment: TRANSPORT: sse MCP_PORT: 8002
```

Q03

Intermediate

Walk through all 10 MCP tools in your server. Categorise them by type (read, write, admin) and explain the design rationale for having exactly these 10.

Hint: 10 tools is a deliberate scope — explain what each covers and why nothing was left out.

Key points to cover:

- READ / QUERY tools (5): governance_query (full LangGraph pipeline), query_metrics (InformationAgent → Databricks), search_knowledge_base (KnowledgeAgent → pgvector), get_metadata (MetadataAgent → Collibra), get_system_status (Redis + agents + token usage)
- WRITE / ACTION tools (3): manage_incidents (CapacityAgent → Jira CRUD), manage_rules (RuleAgent → rule CRUD + evaluate), ingest_document_url (download → chunk → embed → upsert)
- ADMIN / CONTROL tools (2): invalidate_cache (by agent name or 'all'), approve_action (re-invoke graph with approved=True for HITL)
- governance_query is the umbrella tool — calls the full LangGraph pipeline; the other 4 read tools expose individual agents for targeted use
- Why both governance_query AND individual agent tools: Claude may choose to call individual tools in sequence (retrieve metrics, then search docs, then synthesise) rather than the full pipeline — gives the AI client more control
- approve_action is the HITL bridge: Claude Desktop can act as the human approver — reads pending_action, user confirms in Claude chat, tool re-invokes graph
- 10 is enough to be comprehensive without overwhelming Claude's tool selection — too many tools degrades LLM tool choice accuracy

Tool Handler Design

Q04

Intermediate

How is a tool handler structured in your MCP server? Walk through the governance_query tool handler as a concrete example.

Hint: Explain the MCP tool registration pattern and the full handler execution path.

Key points to cover:

- MCP tool registration: `@server.call_tool()` decorator registers a handler function; `@server.list_tools()` registers the tool schema (name, description, inputSchema)
- inputSchema uses JSON Schema format — defines required parameters, types, and descriptions that Claude uses to construct the tool call
- governance_query handler receives: query (str), thread_id (str, optional), user_id (str, optional), data_products (list, optional)
- Handler builds initial_state() from the tool arguments — same factory function used by the FastAPI endpoint
- Calls `get_graph().ainvoke(state, config={'configurable': {'thread_id': thread_id}})` — identical to `_run_graph()` in `app.py`
- Returns list of TextContent objects: `[TextContent(type='text', text=json.dumps(result))]` — MCP response format
- Error handling: try/except around graph invocation; on failure returns TextContent with error message — MCP client sees a text response, not a crash
- Thread safety: each tool call creates a new state object; graph singleton is stateless after compilation — safe for concurrent tool invocations

```
# server.py tool handler pattern
@server.list_tools() async def list_tools() -> list[Tool]:
    return [
        Tool(
            name='governance_query',
            description='Run a full governance query through all agents',
            inputSchema={
                'type': 'object',
                'properties': {
                    'query': {'type': 'string', 'description': 'The governance question'},
                    'thread_id': {'type': 'string', 'description': 'Conversation thread ID'},
                },
                'required': ['query']
            },
        )
    ]
@server.call_tool() async def call_tool(name: str, arguments: dict):
    if name == 'governance_query':
        state = initial_state(query=arguments['query'], ...)
        config = {'configurable': {'thread_id': arguments.get('thread_id', 'default')}}
        result = await get_graph().ainvoke(state, config)
        return [TextContent(type='text', text=json.dumps(result))]
    raise ValueError(f'Unknown tool: {name}')
```

Q05

Intermediate

The approve_action tool is unique — it re-invokes the LangGraph graph with approved=True on an existing thread. Walk through exactly how this enables HITL from Claude Desktop.

Hint: This is the most architecturally interesting MCP tool — explain the full flow.

Key points to cover:

- Scenario: governance_query runs, auto_ticket_node detects anomalies, sets pending_action in state and returns — checkpoint saved to PostgresSaver
- governance_query returns to Claude Desktop: 'Anomaly detected: retention dropped 15%. Pending action: create Jira incident. Thread: abc-123. Approve?'
- User reads in Claude Desktop chat and says 'Yes, approve' — Claude calls approve_action(thread_id='abc-123')
- approve_action handler: loads the thread checkpoint via `get_graph()`, constructs state update with `approved=True`
- Re-invokes: `graph.ainvoke({'approved': True}, config={'configurable': {'thread_id': 'abc-123'}})` — LangGraph rehydrates checkpoint and resumes
- auto_ticket_node runs again: sees `approved=True`, creates Jira ticket, clears pending_action
- approve_action returns: TextContent with Jira ticket ID and URL — Claude Desktop shows confirmation to user
- This makes Claude Desktop itself the HITL interface — no Streamlit UI or Teams bot needed for approval; the AI assistant IS the human-in-the-loop gateway

Q06**Advanced**

The ingest_document_url tool downloads a URL, ingests it, and embeds it into pgvector — all from inside an MCP tool call. What are the security risks of this design?

Hint: A tool that fetches arbitrary URLs is a significant attack surface.

Key points to cover:

- Risk 1: SSRF (Server-Side Request Forgery) — attacker passes internal URL: 'http://169.254.169.254/latest/meta-data/' (AWS instance metadata) — server fetches and returns secrets
 - Risk 2: malicious content injection — URL points to a PDF containing prompt injection instructions — ingested into pgvector, poisons knowledge base
 - Risk 3: large file DoS — URL points to a 10GB file — server downloads entirely before rejecting — memory/disk exhaustion
 - Risk 4: redirect chain — URL redirects to internal service — SSRF via redirect
 - Mitigations: URL allowlist (only https://, block 169.254.x.x, 10.x.x.x, 172.16.x.x, 127.x.x.x private ranges)
 - Content-Length check before download: HEAD request first — reject if Content-Length > 50MB
 - Timeout on download: max 30 seconds — prevents slow-loris style hanging connections
 - Scan ingested content: run guardrail injection check on extracted text before embedding — catches poisoned documents
 - Auth on MCP server: SSE transport should require an API key header — prevent unauthenticated ingest_document_url calls from the public internet
-

MCP Client Factory & USE_MCP Flag

Q07**Intermediate**

Explain core/mcp_client.py and the USE_MCP flag. What does it control and when would you set USE_MCP=true in production?

Hint: The MCP client is the outbound side — your system calling other MCP servers.

Key points to cover:

- core/mcp_client.py creates MCP clients that your application uses to call EXTERNAL MCP servers — the reverse direction from your MCP server
 - USE_MCP=false (dev default): MetadataAgent calls Collibra REST API directly; CapacityAgent calls Jira REST API directly
 - USE_MCP=true (prod option): MetadataAgent routes through an MCP client to a Collibra MCP server; CapacityAgent routes through a Jira MCP server
 - Why would Collibra/Jira expose MCP servers? Vendors are adding MCP support — Collibra's MCP server handles auth, rate limiting, and schema normalisation
 - MCP client factory: mcp_client_factory() reads USE_MCP env var; if true, creates MCP client with server URL; if false, returns None (direct API used)
 - Graceful fallback: if MCP client fails to connect (server down), factory returns None and agents fall back to direct REST — same pattern as Redis fallback
 - Benefit of USE_MCP=true: authentication is delegated to the MCP server — no Collibra API keys in your application; vendor manages auth rotation
 - Current status: USE_MCP=false in both dev and prod for your project — MCP routing for Collibra/Jira is a future enhancement when vendors release stable MCP servers
-

Q08

Gotcha

Your MCP server tests in `test_mcp_server.py` don't require a transport connection — they test tool handlers directly. Why is this possible and what would be missing if you tested via the actual transport?

Hint: Think about how MCP server testing differs from API endpoint testing.

Key points to cover:

- MCP tool handlers are plain async functions decorated with `@server.call_tool()` — they can be called directly in tests without starting the stdio or SSE transport
- `test_mcp_server.py` calls the handler function directly with a dict of arguments: `await call_tool('governance_query', {'query': 'test'})`
- No transport needed: the test bypasses JSON-RPC message framing, transport negotiation, and MCP protocol handshake entirely
- This is equivalent to testing a FastAPI endpoint by calling the view function directly instead of through `TestClient` — faster but less realistic
- What's missing: transport-level tests would catch JSON serialisation errors (non-serialisable result types), MCP protocol version mismatches, tool schema validation (is `inputSchema` valid JSON Schema?)
- What's also missing: Claude Desktop integration test — does Claude actually discover and correctly invoke your tools? Can't test this with unit tests alone
- Practical tradeoff: transport tests require running the full server process — slow, complex; handler tests are fast and catch 90% of bugs
- Best of both: handler tests for logic, one integration smoke test that starts the SSE server and connects a real MCP client — covers the remaining 10%

Production Considerations & Architecture Depth

Q09

Advanced

Your MCP server shares `src/` imports with FastAPI — both call `get_graph()`, both use the same agents. What happens when both run simultaneously in Docker Compose and a request hits both?

Hint: Think about shared state, singleton isolation, and the graph singleton.

Key points to cover:

- Each process (FastAPI on port 8000, MCP server on port 8002) runs independently — no shared memory between processes
- `get_graph()` singleton: each process builds and caches its own `_graph` instance — two separate compiled `LangGraph` graphs in memory
- Redis cache: both processes share the same Redis — a cache entry written by a FastAPI query hit can be returned by an MCP query (correct behaviour, cache is shared)
- `PostgresSaver` checkpoints: both processes share the same Neon DB — `thread_id`-based isolation prevents collision; a FastAPI session and MCP session on the same `thread_id` would interleave (avoid in practice)
- LLM fallback chain: both processes independently call the same LLM providers — token budget counter in Redis is shared, correctly limiting total spend
- No in-memory state sharing: rate limiter counters (if memory://), in-memory cache fallback — these are per-process and siloed
- Operational consideration: two processes = double the memory overhead for graph compilation, agent instances, and service clients
- Improvement: share a single process that serves both FastAPI and MCP SSE — reduces overhead; FastAPI can add an `/mcp` SSE route rather than a separate process

Q10**Advanced**

How would you add authentication to the MCP SSE endpoint in production? stdio doesn't need it, but SSE on port 8002 is network-accessible.

Hint: SSE transport is an HTTP server — standard HTTP auth patterns apply.

Key points to cover:

- Current state: SSE transport has no authentication — any client that can reach port 8002 can call all 10 tools including `ingest_document_url` and `manage_rules`
 - Option 1: API key header — MCP client sends `X-API-Key` header; server validates against a key stored in AWS Secrets Manager
 - Option 2: Bearer token (JWT) — server validates JWT signature and expiry; MCP client includes `Authorization: Bearer {token}`
 - Option 3: VPC restriction — expose port 8002 only within VPC via security group; only ECS tasks and internal services can reach it
 - Option 4: mTLS — mutual TLS where both client and server present certificates; strongest but most complex to configure
 - Implementation: FastAPI-style middleware in the SSE HTTP handler — inspect headers before routing to tool dispatcher
 - Claude Desktop (stdio) needs no auth: it runs as a subprocess with your local credentials; the process-level trust is sufficient
 - For EKS/prod: add an AWS ALB in front of the MCP SSE server with Cognito or API Gateway authoriser — same pattern as the FastAPI ALB
-

Q11**Advanced**

An interviewer asks: 'MCP is a new protocol. What happens to your system if Anthropic changes the MCP spec in a breaking way?' How do you mitigate versioning risk?

Hint: Demonstrate awareness of protocol stability and dependency risk.

Key points to cover:

- MCP version: your `server.py` imports from the `mcp` Python library (pinned in `requirements.txt`); the library handles protocol-level details
 - Breaking change risk: if Anthropic changes the JSON-RPC message format or tool schema structure, the `mcp` library version must be updated
 - Mitigation 1: pin the `mcp` library version (e.g. `mcp==1.2.0`) in `requirements.txt` — don't use `>=`; avoid surprise upgrades in CI
 - Mitigation 2: your tool handlers are pure async functions — they are not tightly coupled to the MCP library internals; only the decorator and `TextContent` return type touch MCP
 - Mitigation 3: abstraction layer — wrap MCP tool registration in your own decorator; if MCP changes, update one place rather than all 10 tool handlers
 - Mitigation 4: canary upgrade — test MCP library upgrades in a separate environment before rolling to prod
 - Mitigation 5: FastAPI API as fallback — all 10 MCP tools have equivalent FastAPI endpoints; if MCP breaks, Claude Desktop can be reconfigured to call the REST API via a thin wrapper
 - Protocol maturity: MCP is stabilising rapidly (2024-2025); Anthropic has strong incentive to maintain backward compat as ecosystem grows — risk is real but manageable
-

An interviewer asks: 'You have FastAPI, MCP server, Streamlit, and Teams bot — four interfaces to the same LangGraph backend. How do you keep them in sync as the system evolves? What breaks when you add a new agent?'

Hint: This is a system evolution / interface proliferation question.

Key points to cover:

- Shared core: all four interfaces call the same `get_graph()` and `_run_graph()` equivalent — adding a new agent to the graph automatically benefits all interfaces
- What doesn't auto-propagate: MCP tool `list_tools()` is hardcoded — adding a new agent doesn't add a new MCP tool automatically; must update `server.py`
- What doesn't auto-propagate: FastAPI `/agents/status` endpoint lists agents manually — must update the agent list
- What doesn't auto-propagate: Streamlit HITL panel hardcodes agent names for display — must update the UI
- Mitigation: agent registry — a central list of agent names and metadata in `config/settings.py` (`DATA_PRODUCTS` is a similar pattern); all interfaces read from this registry
- MCP auto-discovery: generate `list_tools()` dynamically from the agent registry — each registered agent gets a corresponding MCP tool automatically
- Testing: adding a new agent requires `test_new_agent.py` (injection pattern) AND smoke test coverage — `test_day20_smoke.py` must be extended
- Interface contract: define an `AgentManifest` (`name`, `description`, `input_fields`, `output_fields`) — MCP `inputSchema`, FastAPI docs, and Streamlit UI all derive from it
- This is a real maintenance challenge worth raising proactively — shows maturity beyond 'it works in a demo'